

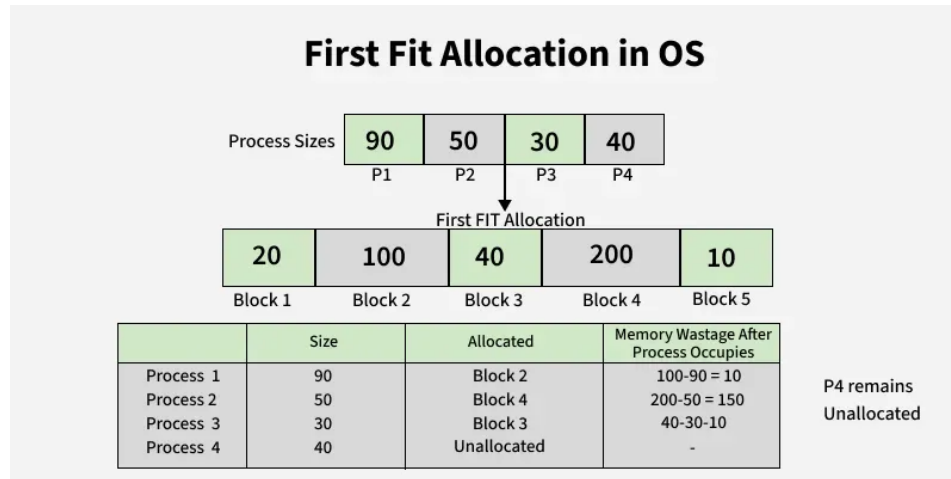


Trabalho Prático II (TP II) - Listas e Métodos de Ordenação

- Submissão com data e hora de entrega disponíveis na plataforma da disciplina. O que vale é o horário do Moodle, e não do *seu*, ou do *meu* relógio!!!
- Clareza, identificação e comentários no código também vão valer pontos. Por isso, escolha cuidadosamente o nome das variáveis e torne o código o mais legível possível.
- O padrão de entrada e saída deve ser respeitado exatamente como determinado no enunciado. Parte da correção é automática, não respeitar as instruções enunciadas pode acarretar em perda de pontos.
- Durante a correção, os programas serão submetidos a vários casos de testes, com características variadas.
- A avaliação considerará o tempo de execução e o percentual de respostas corretas.
- Eventualmente serão realizadas entrevistas sobre o trabalho para complementar a avaliação;
- O trabalho é em grupo de até 2 (duas) pessoas.
- Será aceito trabalhos após a data de entrega, todavia com um decréscimo de 0,05 a cada 10min.
- Os códigos fonte serão submetidos a uma ferramenta de detecção de plágios em software.
- Códigos cuja autoria não seja do aluno, com alto nível de similaridade em relação a outros trabalhos, ou que não puder ser explicado, acarretará na perda da nota.
- Códigos ou funções prontas específicas de algoritmos para solução dos problemas elencados não são aceitos
- Não serão considerados algoritmos parcialmente implementados.
- Procedimento para a entrega:.
 1. Submissão: via **Moodle**.
 2. Os nomes dos arquivos e das funções devem ser especificados considerando boas práticas de programação.
 3. Funções auxiliares, complementares aquelas definidas, podem ser especificadas e implementadas, se necessário.
 4. A solução deve ser devidamente modularizada e separar a especificação da implementação em arquivos *.h* e *.c* sempre que cabível.
 5. Os arquivos a serem entregues, incluindo aquele que contém *main()*, devem ser compactados (*.zip*), sendo o arquivo resultante submetido via **Moodle**.
 6. Você deve submeter os arquivos *.h*, *.c* e o *.pdf* (relatório) na raiz do arquivo *.zip*. Use os nomes dos arquivos *.h* e *.c* exatamente como pedido.
 7. Caracteres como acento, cedilha e afins não devem ser utilizados para especificar nomes de arquivos ou comentários no código.
- **Bom trabalho!**

Alocador de memória baseado em first-fit

Em um sistema operacional, o gerenciamento de memória é um ponto crítico para o bom funcionamento. Diversos algoritmos são implementados com o intuito de otimizar essa alocação, como *best-fit*, *first-fit*, etc. No presente trabalho, seu intuito é simular essa alocação de memória através do algoritmo guloso *first-fit*. Nele, a alocação de memória ocorre no primeiro espaço de memória disponível, se mantendo até que ocorra a desalocação.



Imposições e comentários gerais

Neste trabalho, as seguintes regras devem ser seguidas:

- Seu programa não pode ter *memory leaks*, ou seja, toda memória alocada pelo seu código deve ser corretamente liberada antes do final da execução. (Dica: utilize a ferramenta *valgrind* para se certificar de que seu código libera toda a memória alocada)
- Um grande número de *Warnings* ocasionará a redução na nota final.

O que deve ser entregue

- Código fonte do programa em C (**bem indentado e comentado**).
- Documentação do trabalho (relatório¹). A documentação deve conter:
 1. **Implementação:** descrição sobre a implementação do programa. Não faça “*print screens*” de telas. Ao contrário, procure resumir ao máximo a documentação, fazendo referência ao que julgar mais relevante. É importante, no entanto, que seja descrito o funcionamento das principais funções e procedimentos utilizados, bem como decisões tomadas relativas aos casos e detalhes de especificação que porventura estejam omissos no enunciado. Muito importante: os códigos utilizados na implementação devem ser inseridos na documentação.
 2. **Impressões gerais:** descreva o seu processo de implementação deste trabalho. Aponte coisas que gostou bem como aquelas que o desagradou. Avalie o que o motivou, conhecimentos que adquiriu, entre outros.
 3. **Análise:** deve ser feita uma análise dos resultados obtidos com este trabalho.
 4. **Conclusão:** comentários gerais sobre o trabalho e as principais dificuldades encontradas em sua implementação.
 5. **Formato:** PDF.

¹Exemplo de relatório: <https://www.overleaf.com/latex/templates/modelo-relatorio/vprmcsgmcdg>.

Como deve ser feita a entrega

Verifique se seu programa compila e executa na linha de comando antes de efetuar a entrega. Quando o resultado for correto, entregue via *Moodle* até a data disponível na plataforma de entrega um arquivo **.ZIP** com o nome e sobrenome do aluno. Esse arquivo deve conter: (i) os arquivos *.c* e *.h* utilizados na implementação, (ii) instruções de como compilar e executar via terminal, e (iii) o relatório em **PDF**.

Detalhes da implementação

Para atingir o seu objetivo, você deverá implementar um Tipo Abstrato de Dados para representar um Processo sendo alocado, um Evento de alocação ou desalocação e um intervalo para representar um bloco de memória livre. Além disso, as seguintes operações sobre os TAD's devem ser implementadas:

1. `alocar_processo`: aloca um processo na memória.
2. `desalocar_processo`: desaloca um processo da memória.
3. `juntar_intervalos`: após desalocar um processo da memória, quaisquer intervalos contíguos devem ser unidos em um único intervalo.
4. `ordena_eventos`: função para ordenar os eventos de alocar e desalocar processos.
5. `first_fit`: função para alocar os processos na memória.

Os processos serão organizados em eventos de alocação e desalocação, representados a partir de uma lista sobre o TAD Evento. Na função `ordena_eventos` você poderá escolher um dos métodos de ordenação vistos em aula. Contudo, é necessário justificar a escolha do método no relatório. Em caso de empate de tempo entre eventos de alocar e desalocar, a preferência é para desalocar a memória.

A memória deverá ser implementada como uma lista encadeada de intervalos, que representam blocos livres de memória. O tamanho inicial da memória é dado pela soma do tamanho ocupado por todos os processos, representado inicialmente por um intervalo único que começa na posição zero.

O TAD deve ser implementado utilizando a separação da interface no *.h* e implementação *.c* discutida em sala, bem como as convenções de tradução. Caso a operação possa dar errado, devem ser definidos retornos com erro, tratados no corpo principal.

O código-fonte deve ser modularizado corretamente em três arquivos: *tp.c*, *aluno.h* e *aluno.c*. Arquivos de implementação base serão fornecidos, que não devem ser excluídos apenas incrementados. A separação das operações em funções e procedimentos está a cargo do aluno, porém, **não deve haver acúmulo** de operações dentro de uma mesma função/procedimento.

Entrada

A entrada é dada por meio do terminal e é composta de vários conjuntos de teste. Para facilitar, a entrada será fornecida por meio de arquivos.² A primeira linha de um conjunto de testes contém um inteiro N que representa a quantidade de processos a serem lidos. Portanto, as N linhas seguintes contém, cada uma, a descrição de um processo. Um processo é descrito por três valores A , D e Tam que representam, respectivamente, o instante de tempo de alocação, o instante de tempo em que o processo deverá ser desalocado e tamanho ocupado pelo processo.

Saída

Para cada teste seu programa deve imprimir ao final o log de alocações dos processos, indicando em qual espaço de memória um processo foi alocado.

²Para usar o arquivo como entrada no terminal, utilize `./executavel < nome_do_arquivo_de_teste`.

Exemplo de um caso de teste

Exemplo da saída esperada dada uma entrada:

Entrada	Saída
5	Processo 0 alocado na posição 0
1 10 10	Processo 1 alocado na posição 10
3 6 5	Processo 2 alocado na posição 15
4 5 9	Processo 3 alocado na posição 10
7 12 12	Processo 4 alocado na posição 22
9 11 5	

Diretivas de Compilação

As seguintes diretivas de compilação devem ser usadas (essas são as mesmas usadas no run.codes).

```
$ gcc -c aluno.c -Wall
$ gcc -c tp.c -Wall
$ gcc aluno.o tp.o -o exe
```

Avaliação de *leaks* de memória

Uma forma de avaliar se não há *leaks* de memória é usando a ferramenta *valgrind*. O *valgrind* é um *framework* de instrumentação para análise dinâmica de um código e é muito útil para resolver dois problemas em seus programas: **vazamento de memória e acesso a posições inválidas de memória** (o que pode levar a *segmentation fault*). Um exemplo de uso é:

```
1 gcc -g -o exe *.c -Wall
2 valgrind --leak-check=full -s ./exe < casoteste.in
```

Espera-se uma saída com o fim semelhante a:

```
1 ==xxxxxx== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Para instalar no Linux, basta usar: `sudo apt install valgrind`.